

CEE6501 — Lecture 13.1

Force Density Method (Linear Examples)

Learning Objectives

By the end of this lecture, you will be able to:

- recall the force density formulation and its linear structure
- explain why the force density method leads to a linear system of equations
- implement the force density method in code for 2D and 3D problems
- interpret how different force density patterns influence structural form
- apply the method to simple and complex cable-net structures
- extend the formulation to include out-of-plane behavior and applied loads

Agenda

- Part 1 — Review of force density method (linear formulation)
- Part 2 — Code implementation and workflow
- Part 3 — Schek 2D structure examples
- Part 4 — Complex 2D cable-net structures
- Part 5 — Extension to 3D form finding
- Part 6 — Adding applied loads to the system

Part 1 - Review of Force Density (Linear)

Define

$$\mathbf{D} = \mathbf{C}^T \mathbf{Q} \mathbf{C}, \quad \mathbf{D}_f = \mathbf{C}_f^T \mathbf{Q} \mathbf{C}_f$$

Doing this for all 3 directions, the force density equations become

$$\mathbf{D}\mathbf{x} = \mathbf{p}_x - \mathbf{D}_f\mathbf{x}_f \quad (4-1)$$

$$\mathbf{D}\mathbf{y} = \mathbf{p}_y - \mathbf{D}_f\mathbf{y}_f \quad (4-2)$$

$$\mathbf{D}\mathbf{z} = \mathbf{p}_z - \mathbf{D}_f\mathbf{z}_f \quad (4-3)$$

These are **linear equations in the unknown coordinates**.

The Key Advantage: A Linear Formulation

For prescribed:

- connectivity
- support coordinates
- nodal loads
- force densities

the unknown free-node coordinates follow from a **single linear solve**.

That was the original appeal of the force density method.

In a planar case, once we solve

$$\mathbf{x} = \mathbf{D}^{-1} (\mathbf{p}_x - \mathbf{D}_f \mathbf{x}_f) \quad (5-1)$$

$$\mathbf{y} = \mathbf{D}^{-1} (\mathbf{p}_y - \mathbf{D}_f \mathbf{y}_f) \quad (5-2)$$

and for 3D we add in

$$\mathbf{z} = \mathbf{D}^{-1} (\mathbf{p}_z - \mathbf{D}_f \mathbf{z}_f) \quad (5-3)$$

Once $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ are solved, the new equilibrium geometry is fully determined.

From there, we can:

1. reconstruct the updated nodal geometry
2. compute the corresponding branch lengths
3. recover the branch forces from

$$\mathbf{s} = \mathbf{L}\mathbf{q}$$

This is why the force density method is often described as a **one-step method**: once the force densities $\mathbf{q}$ are prescribed, the equilibrium geometry follows from a linear solve.

Solution Procedure

For a simple planar example with two free nodes, the workflow is:

1. choose the connectivity and build $\mathbf{C}$ and $\mathbf{C}_f$
2. prescribe the force densities and build $\mathbf{Q}$ (user chooses)
3. compute

$$\mathbf{D} = \mathbf{C}^T \mathbf{Q} \mathbf{C}$$

and

$$\mathbf{D}_f = \mathbf{C}^T \mathbf{Q} \mathbf{C}_f$$

4. solve separately for the unknown x - and y -coordinates
5. recover branch lengths and then branch forces

Part 2 - Code Implementation

Repository Link

<https://github.com/Bruun-Automation-Research-Lab/matrix-form-finding>

Running Code

Run from the `./formfinder/main.py` file

Text data saved to `./debug` folder

Output plots and daya saved to Text data saved to `./output` folder

Running from the `main.py` File

To run the linear force density solver, open `main.py` and set:

- `solver=solvers[1]` for `FD_linear`
- `structure=structs[...]` to whichever example you want to run


```
if __name__ == "__main__":  
    solvers = {  
        1: "FD_linear",  
        2: "FD_nonlinear",  
        3: "SM",  
        4: "DR_imp",  
        5: "DR_leap",  
    }  
    structs = {  
        1: "Schek_2D",  
        2: "Schek_Modified",  
        3: "Veenendaal",  
        4: "Pauletti",  
        5: "CEE6501_2D",  
        6: "CEE6501_PointLoad",  
        7: "Star",  
    }  
    run = FormFinder(  
        solver=solvers[1], structure=structs[1], debug=True, plot_save=True  
    )  
    run.solve()
```

Input File Structure

Input files specified in `./formfinder/structures` folder

Define as a function you can import

```

7  def generate_struct():
8      # -----
9      # Node coordinates
10     # Outer support nodes (6-9) are fixed.
11     # Inner nodes (1-5) define the central network.
12     # All nodes lie in the XY plane, so z = 0 for every node.
13     # -----
14     nodes = {
15         1: (-2.0, 0.0, 0.0),
16         2: (0.0, -2.0, 0.0),
17         3: (2.0, 0.0, 0.0),
18         4: (0.0, 2.0, 0.0),
19         5: (0.0, 0.0, 0.0),
20         6: (-3.0, 0.0, 0.0),
21         7: (0.0, -3.0, 0.0),
22         8: (3.0, 0.0, 0.0),
23         9: (0.0, 3.0, 0.0),
24     }
25
26     # -----
27     # Element connectivity
28     # Format: {element_id: (start_node, end_node)}
29     # -----
30     elements = {
31         1: (1, 6),
32         2: (2, 7),
33         3: (3, 8),
34         4: (4, 9),
35         5: (1, 4),
36         6: (1, 2),
37         7: (2, 3),
38         8: (3, 4),
39         9: (4, 5),
40         10: (1, 5),
41         11: (2, 5),
42         12: (3, 5),
43     }
44 
```

```

45 # -----
46 # Initial element preload / force-density values
47 # In FD_linear, this is the prescribed q value for each element.
48 # -----
49 elements_preload = {
50     1: 1.0,
51     2: 1.0,
52     3: 1.0,
53     4: 1.0,
54     5: 1.0,
55     6: 1.0,
56     7: 1.0,
57     8: 1.0,
58     9: 1.0,
59     10: 1.0,
60     11: 1.0,
61     12: 1.0,
62 }
63
64 # -----
65 # Applied nodal loads
66 # Format: {node_id: (Fx, Fy, Fz)}
67 # No external loads are applied in this example.
68 # -----
69 nodes_loads = {
70     1: (0.0, 0.0, 0.0),
71     2: (0.0, 0.0, 0.0),
72     3: (0.0, 0.0, 0.0),
73     4: (0.0, 0.0, 0.0),
74     5: (0.0, 0.0, 0.0),
75     6: (0.0, 0.0, 0.0),
76     7: (0.0, 0.0, 0.0),
77     8: (0.0, 0.0, 0.0),
78     9: (0.0, 0.0, 0.0),
79 }
80
81 # -----
82 # Boundary conditions
83 # 0 = free
84 # 1 = fixed
85 # -----
86 nodes_fixed = {
87     1: 0,
88     2: 0,
89     3: 0,
90     4: 0,
91     5: 0,
92     6: 1,
93     7: 1,
94     8: 1,
95     9: 1,
96 }
97
98 return nodes, elements, elements_preload, nodes_loads, nodes_fixed
99

```

Why Define as Function?

Allows you to create more complex definition, rather than hard-coding all information

For example, defining nodal coordinates in the `CEE6501_PointLoad.py` structure

```
def generate_struct(rows=8, cols=10, spacing=1.0):
    # -----
    # Initialize dictionaries
    # -----
    nodes = {}
    # -----
    # Node coordinates
    # rows x cols rectangular grid in the XY plane with uniform spacing.
    # All nodes start with z = 0.0.
    # Node numbering proceeds row by row:
    # Left to right, then bottom to top.
    # -----
    node_id = 1
    for i in range(rows):
        for j in range(cols):
            nodes[node_id] = (j * spacing, i * spacing, 0.0)
            node_id += 1
```

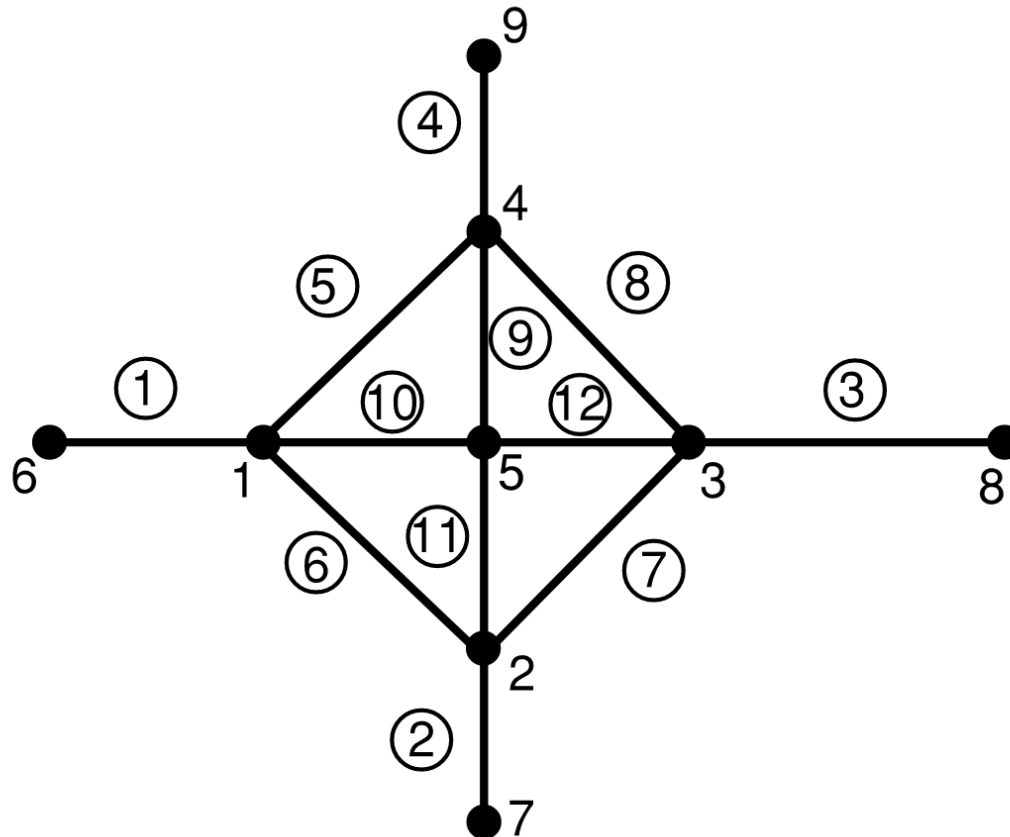
Run Flags

```
run = FormFinder(  
    solver=solvers[1], structure=structs[1], debug=True, plot_save=True  
)
```

- `debug=True`
writes a detailed text log of key variables and results throughout the analysis and iterations to the `./debug` folder.
Useful for understanding solver behavior, but can slow the run significantly for larger structures.
- `plot_save=True`
saves the generated plots to the `./output` folder.s

Part 3 - Schek Paper 2D Structure

Description of Structure



Geometry Setup (Output in Debug)

NODES [n x 3]				ELEMENTS [m x 2] & [m x 1]				NODAL LOADS [n x 3]				NODAL FIXITY [n x 1]			
		x	y			n_i	n_j			p_x	p_y	p_z	1 = fixed		0 = free
		1 x	1 x			1 x	1 x			1 x	1 x	1 x		fix?	
1		-2.00	0.00		1	1	6	1		0.00	0.00	0.00		1 x	
2		0.00	-2.00		2	2	7	2		0.00	0.00	0.00	1		0
3		2.00	0.00		3	3	8	3		0.00	0.00	0.00	2		0
4		0.00	2.00		4	4	9	4		0.00	0.00	0.00	3		0
5		0.00	0.00		5	1	4	5		0.00	0.00	0.00	4		0
6		-3.00	0.00		6	1	2	6		0.00	0.00	0.00	5		0
7		0.00	-3.00		7	2	3	7		0.00	0.00	0.00	6		1
8		3.00	0.00		8	3	4	8		0.00	0.00	0.00	7		1
9		0.00	3.00		9	4	5	9		0.00	0.00	0.00	8		1
					10	1	5						9		1
					11	2	5								
					12	3	5								

Geometry Setup (Branch-Node Matrices)

BRANCH-NODE MATRIX

[m x n]

C_{total} =

		1	2	3	4	5	6	7	8	9
		1	2	3	4	5	6	7	8	9
1		1	.	.	.	.	-1	.	.	.
2		.	1	.	.	.	.	-1	.	.
3		.	.	1	.	.	.	.	-1	.
4		.	.	.	1	.	.	.	.	-1
5		1	.	.	-1	.	.	.	.	.
6		1	-1	.	.	.	.	.	.	.
7		.	1	-1	.	.	.	.	.	.
8		.	.	1	-1	.	.	.	.	.
9		.	.	.	1	-1	.	.	.	.
10		1	.	.	.	-1	.	.	.	.
11		.	1	.	.	-1	.	.	.	.
12		.	.	1	.	-1	.	.	.	.



BRANCH-NODE MATRIX (FREE)

[m x n_{free}]

C =

		1	2	3	4	5
		1	2	3	4	5
1		1	.	.	.	.
2		.	1	.	.	.
3		.	.	1	.	.
4		.	.	.	1	.
5		1	.	.	-1	.
6		1	-1	.	.	.
7		.	1	-1	.	.
8		.	.	1	-1	.
9		.	.	.	1	-1
10		1	.	.	.	-1
11		.	1	.	.	-1
12		.	.	1	.	-1

BRANCH-NODE MATRIX (FIXED)

[m x n_{fixed}]

C_f =

		1	2	3	4
		6	7	8	9
1		-1	.	.	.
2		.	-1	.	.
3		.	.	-1	.
4		.	.	.	-1
5		.	.	.	.
6		.	.	.	.
7		.	.	.	.
8		.	.	.	.
9		.	.	.	.
10		.	.	.	.
11		.	.	.	.
12		.	.	.	.

Option 1 — Uniform Force Density

Assume the same force density for every member: $q_i = 1$

So the force density vector becomes

$$\mathbf{q} = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]^T$$

Using this, we form the matrices

$$\mathbf{D} = \mathbf{C}^T \mathbf{Q} \mathbf{C} \quad \mathbf{D}_f = \mathbf{C}^T \mathbf{Q} \mathbf{C}_f$$

D (FREE)

[n_free x n_free]

D =

1 x

			1	2	3	4	5
			1	2	3	4	5
1	1		4.00	-1.00	.	-1.00	-1.00
2	2		-1.00	4.00	-1.00	.	-1.00
3	3		.	-1.00	4.00	-1.00	-1.00
4	4		-1.00	.	-1.00	4.00	-1.00
5	5		-1.00	-1.00	-1.00	-1.00	4.00

D (FIXED)

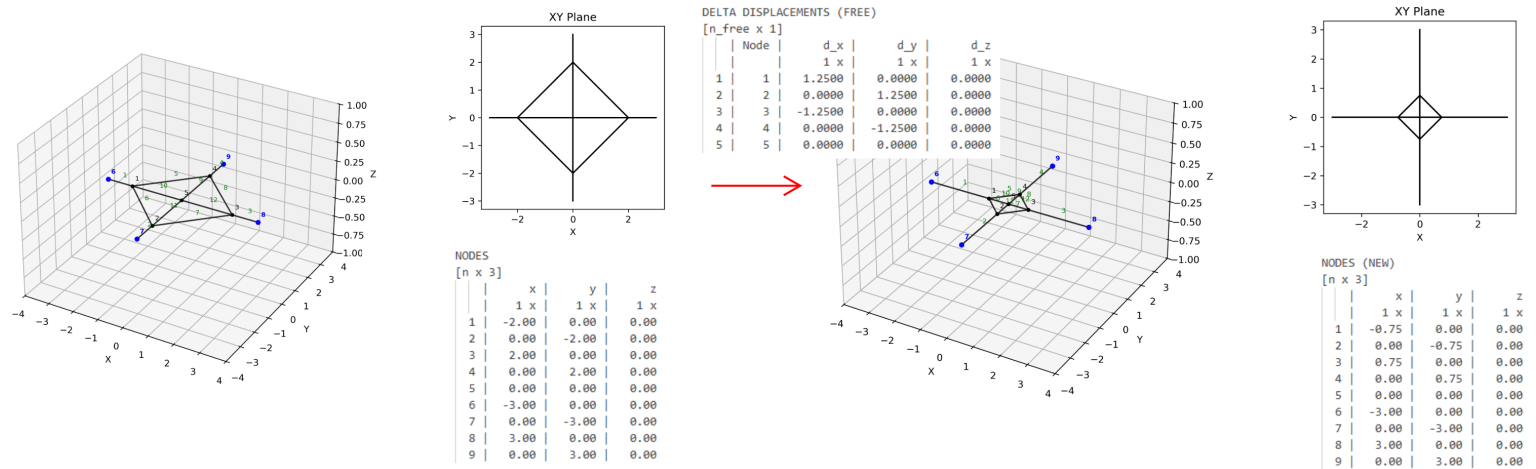
[n_free x n_fixed]

D_f =

1 x

			1	2	3	4
			6	7	8	9
1	1		-1.00	.	.	.
2	2		.	-1.00	.	.
3	3		.	.	-1.00	.
4	4		.	.	.	-1.00
5	5		.	.	.	.

Results (Start -> Final)



Final Node Positions and Branch Forces & Lengths

FINAL NODES

[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-0.750	-0.000	0.000
2	-0.000	-0.750	0.000
3	0.750	-0.000	0.000
4	-0.000	0.750	0.000
5	-0.000	-0.000	0.000
6	-3.000	0.000	0.000
7	0.000	-3.000	0.000
8	3.000	0.000	0.000
9	0.000	3.000	0.000

FINAL FORCE/LENGTH/FORCE DENSITY

[m x 1]

	Force	Length	q
	1 x	1 x	1 x
1	2.2500	2.2500	1.0000
2	2.2500	2.2500	1.0000
3	2.2500	2.2500	1.0000
4	2.2500	2.2500	1.0000
5	1.0607	1.0607	1.0000
6	1.0607	1.0607	1.0000
7	1.0607	1.0607	1.0000
8	1.0607	1.0607	1.0000
9	0.7500	0.7500	1.0000
10	0.7500	0.7500	1.0000
11	0.7500	0.7500	1.0000
12	0.7500	0.7500	1.0000

Option 2 — Scaled Force Density

Scale all force densities for all members uniformly: $q_i = 5$

So the force density vector becomes

$$\mathbf{q} = [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5]^T$$

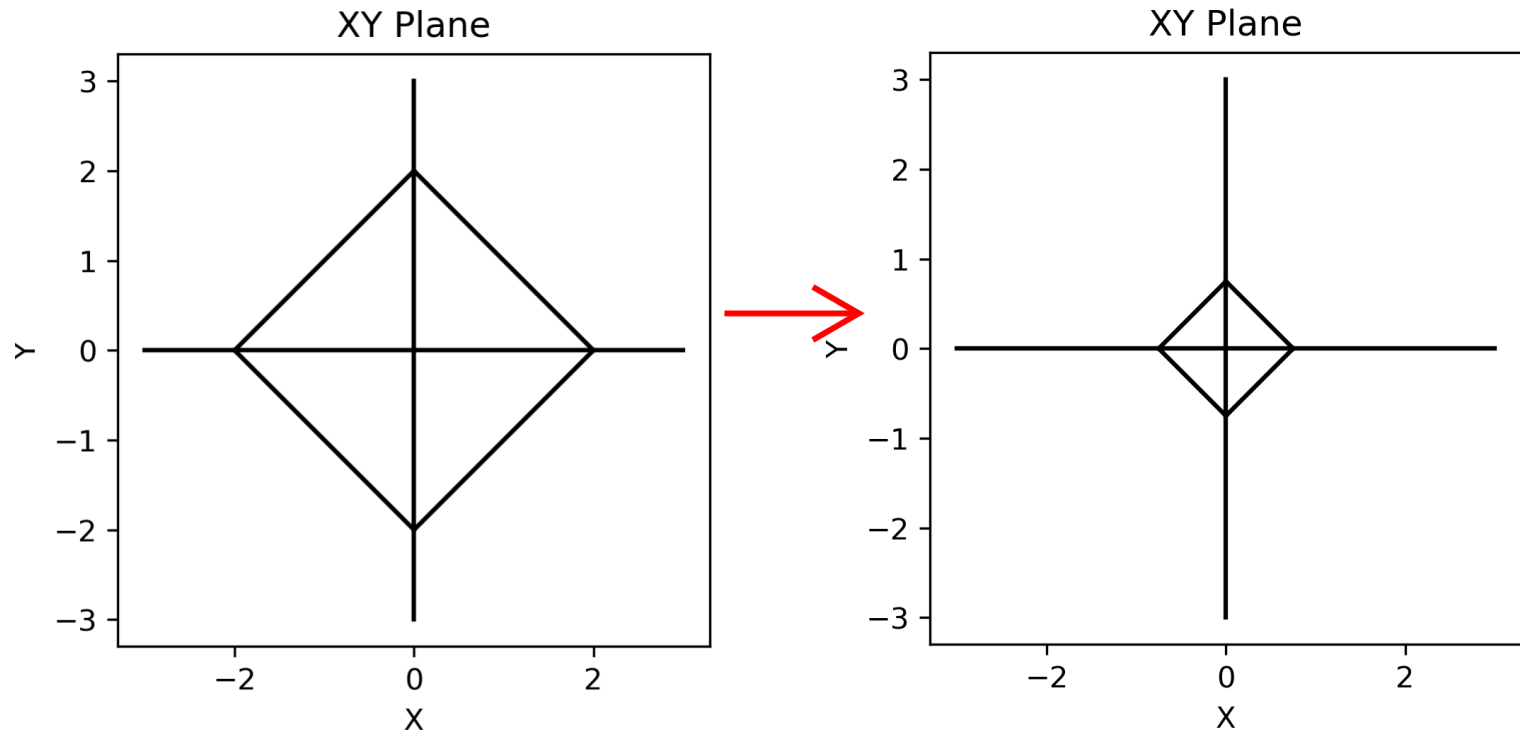
This gives the same *form* as Option 1.

Key idea:

Only the *relative* values of q matter — not their absolute magnitude.

Scaling all q by a constant changes the force level, but not the geometry.

Results (Start -> Final)



Final Node Positions and Branch Forces & Lengths

Same as in the $q = 1$ case!

FINAL NODES

[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-0.750	-0.000	0.000
2	-0.000	-0.750	0.000
3	0.750	-0.000	0.000
4	-0.000	0.750	0.000
5	-0.000	-0.000	0.000
6	-3.000	0.000	0.000
7	0.000	-3.000	0.000
8	3.000	0.000	0.000
9	0.000	3.000	0.000

FINAL FORCE/LENGTH/FORCE DENSITY

[m x 1]

	Force	Length	q
	1 x	1 x	1 x
1	11.2500	2.2500	5.0000
2	11.2500	2.2500	5.0000
3	11.2500	2.2500	5.0000
4	11.2500	2.2500	5.0000
5	5.3033	1.0607	5.0000
6	5.3033	1.0607	5.0000
7	5.3033	1.0607	5.0000
8	5.3033	1.0607	5.0000
9	3.7500	0.7500	5.0000
10	3.7500	0.7500	5.0000
11	3.7500	0.7500	5.0000
12	3.7500	0.7500	5.0000

Option 3 — Non-Uniform Force Density

Assign different force density pattern to selected members:

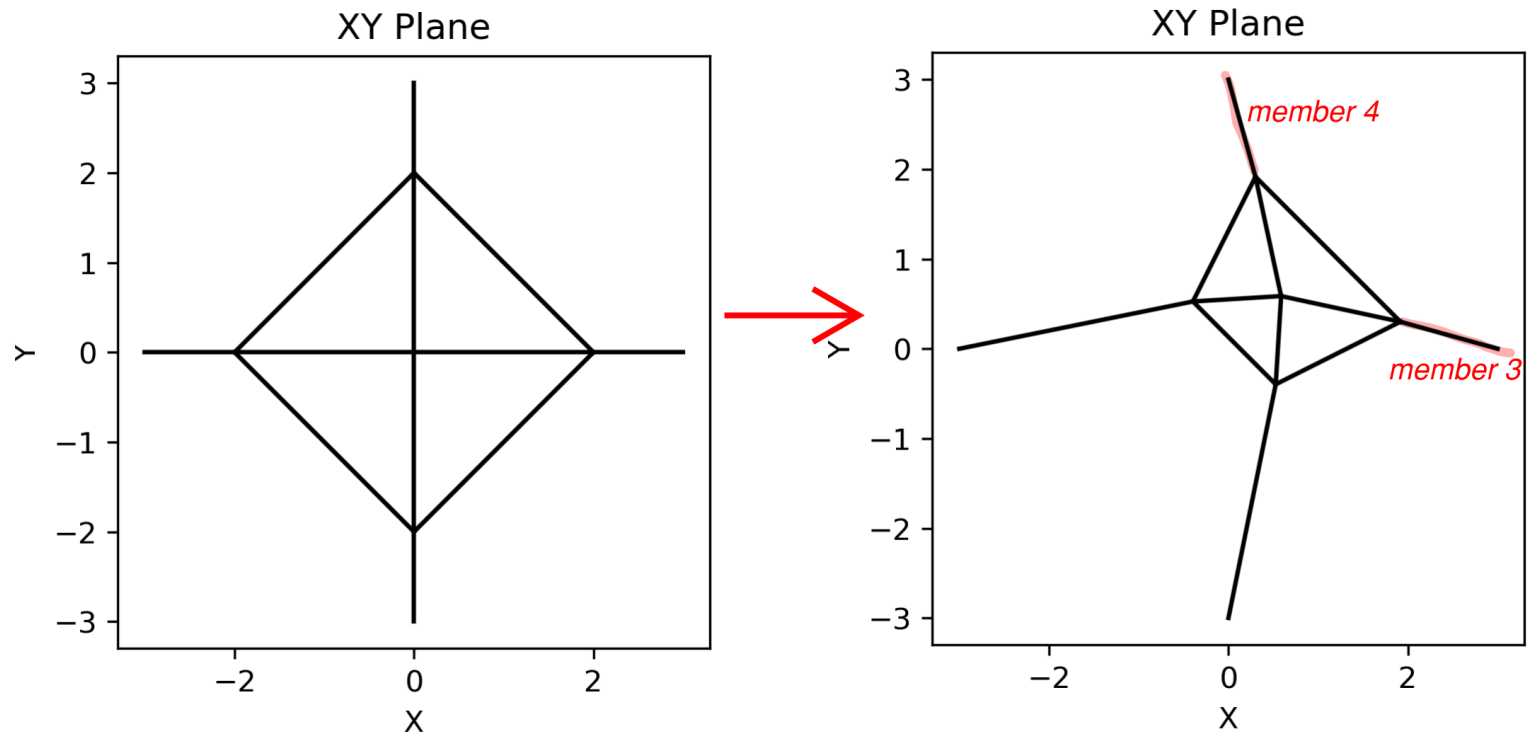
$$q_i = \begin{cases} 4 & \text{for members 3 and 4} \\ 1 & \text{otherwise} \end{cases}$$

This changes the *relative* force densities.

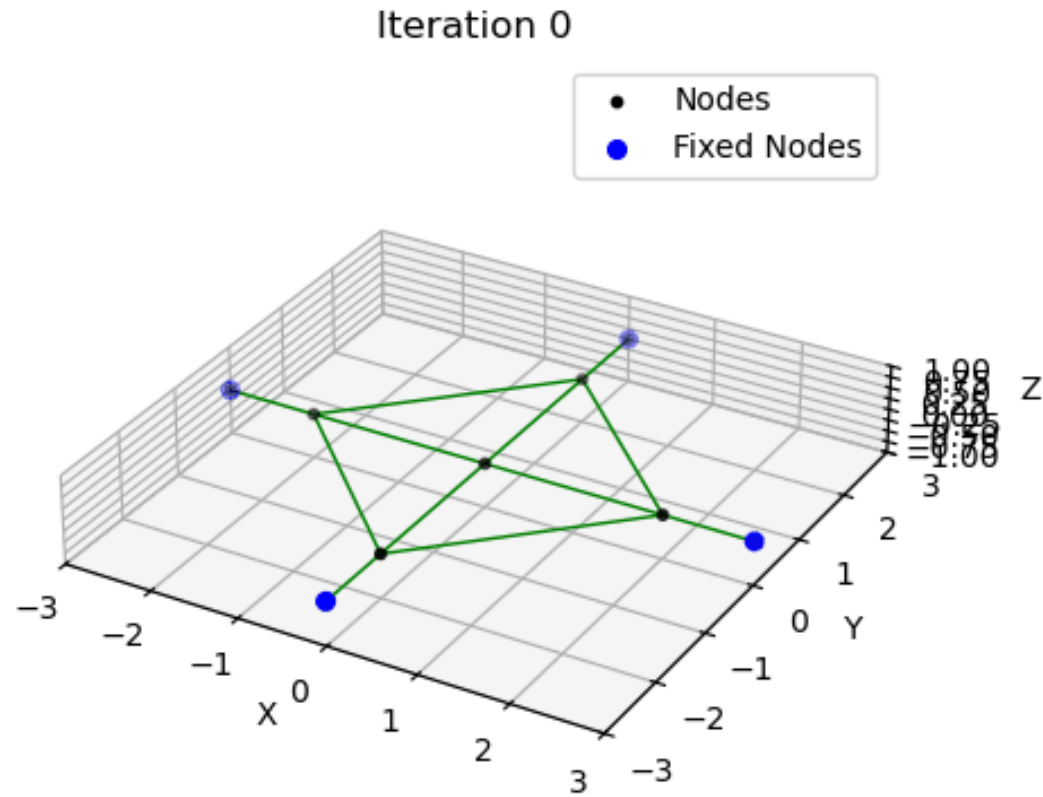
Key idea:

The structure adapts its geometry based on relative q values — members with higher q tend to become shorter and stiffer in the final form.

Results (Start -> Final)



Animation (Only Visible in Jupyter Notebook)

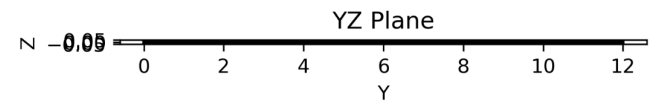
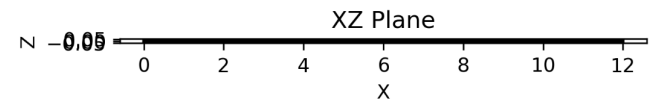
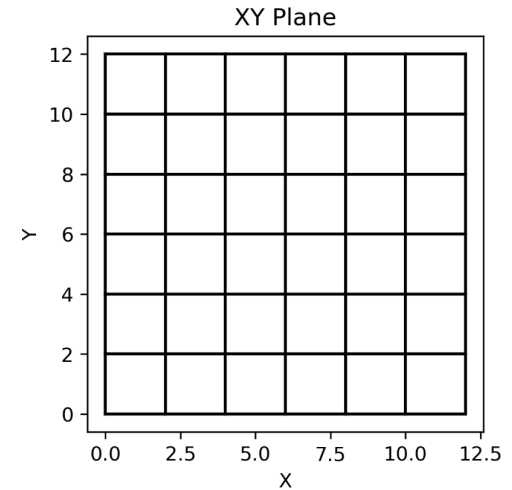
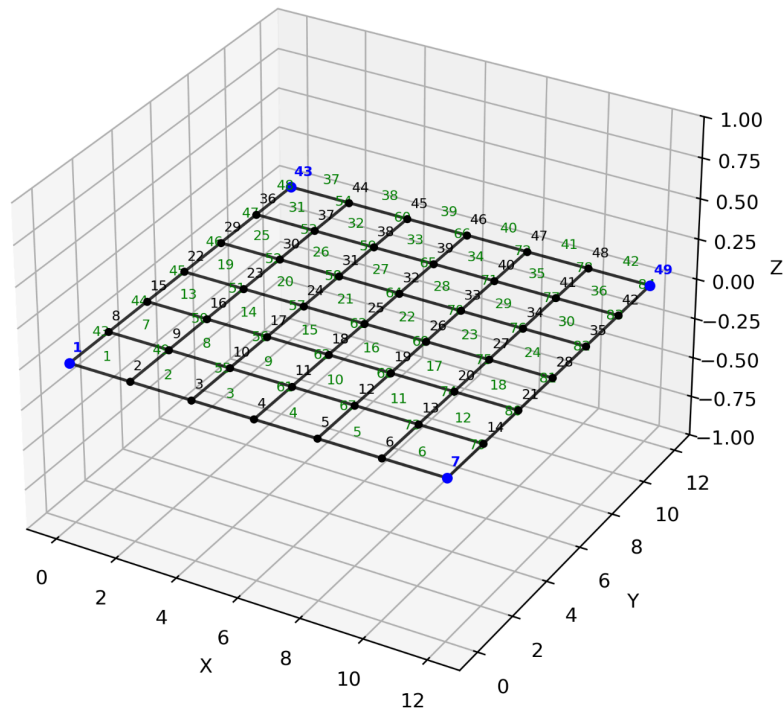


Option 4 — Other Force Density Pattern

Try assigning your own force density pattern. What sort of net structure do you get?

Part 4 - CEE6501 Complex 2D Structure

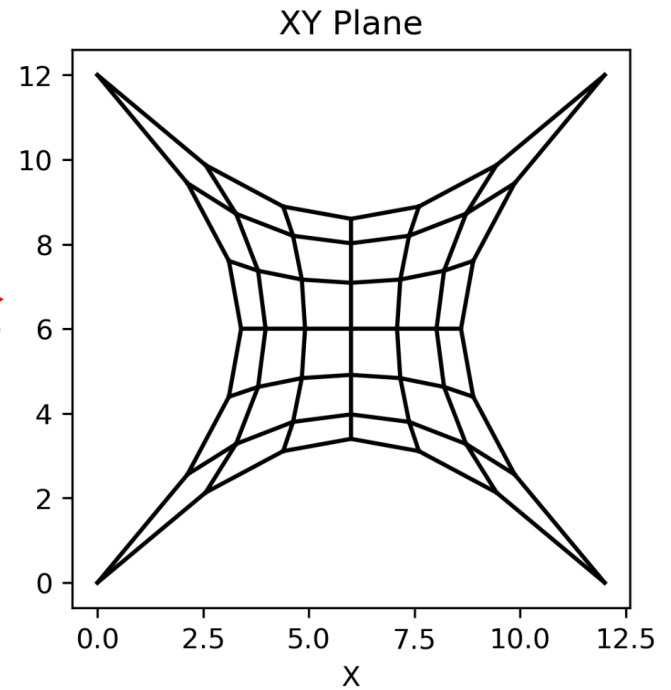
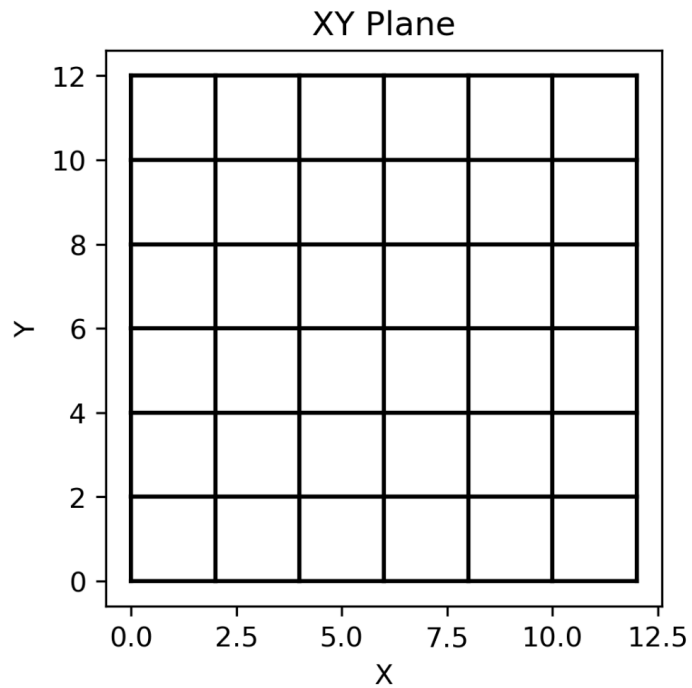
Description of Structure



Option 1 — Uniform Force Density

Assume the same force density for every member: $q_i = 1$

Results (Start -> Final)



Option 2 — Edge-Dominated Force Density

Assign larger force densities to the edge members than to the interior members:

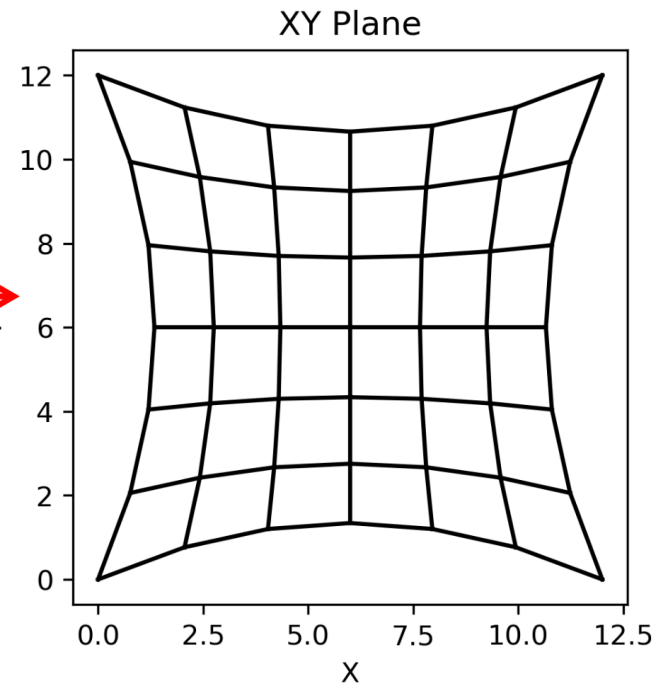
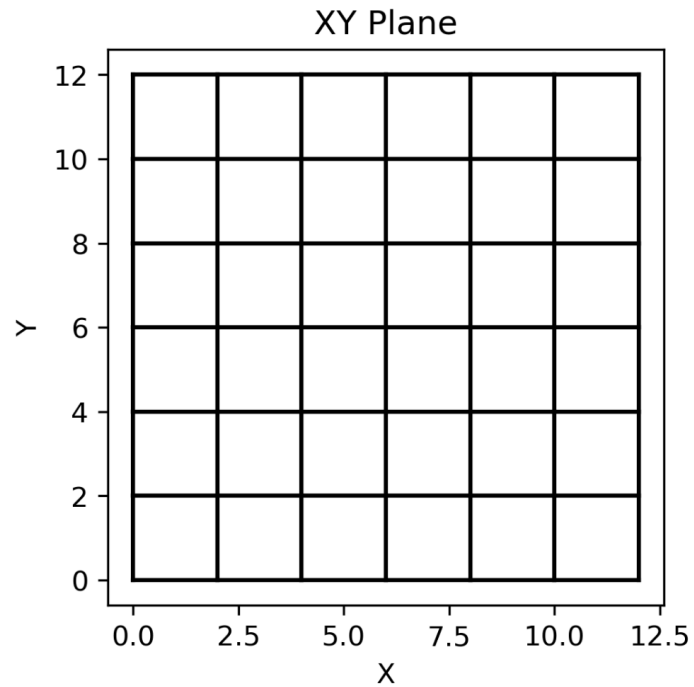
$$q_{\text{edge}} = 5 q_{\text{interior}}$$

So the ratio is

$$q_{\text{edge}} : q_{\text{interior}} = 5 : 1$$

Remember that only the relative values matter, what do you think making the edges stiffer than the interior will do?

Results (Start -> Final)



Option 3 — Interior-Dominated Force Density

Assign larger force densities to the edge members than to the interior members:

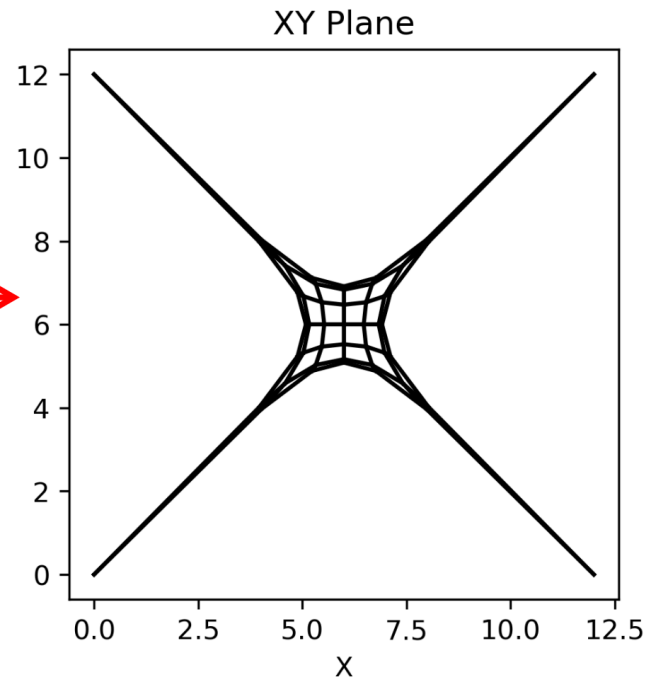
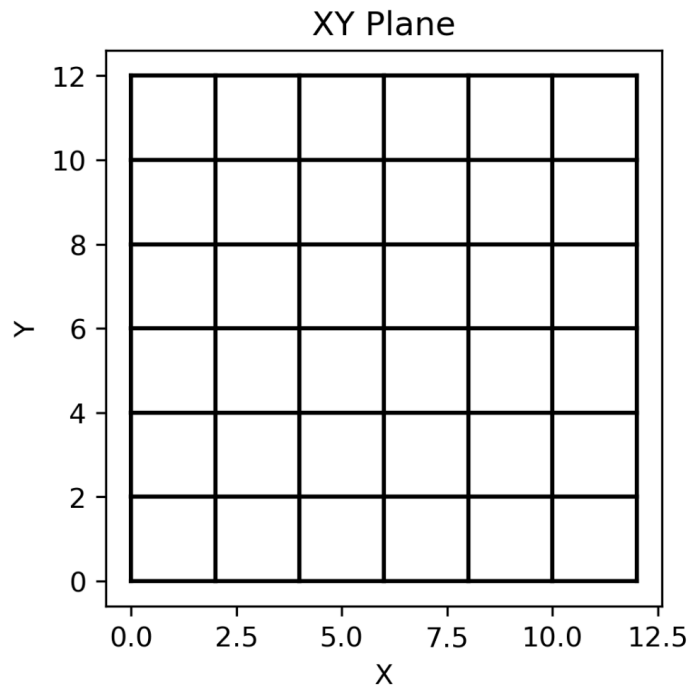
$$q_{\text{interior}} = 5 q_{\text{edge}}$$

So the ratio is

$$q_{\text{interior}} : q_{\text{edge}} = 5 : 1$$

Remember that only the relative values matter, what do you think making the interior stiffer than the edge will do?

Results (Start -> Final)



Part 5 - Extend Force Density to 3D

Extension to 3D

For 3D form finding, we solve for the out-of-plane coordinates:

$$\mathbf{z} = \mathbf{D}^{-1} (\mathbf{p}_z - \mathbf{D}_f \mathbf{z}_f)$$

This is directly analogous to the x and y equations.

Key idea:

The same matrix $\mathbf{D}$ and $\mathbf{D}_f$ are used — we simply solve an additional system for the z -direction.

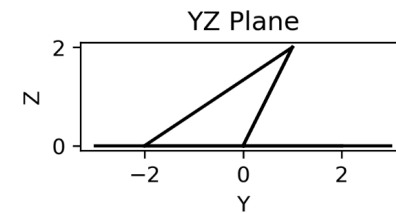
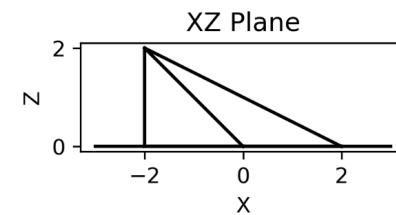
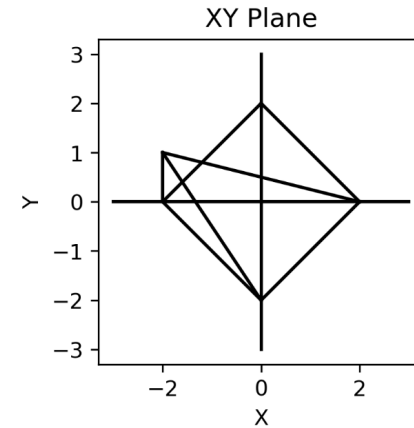
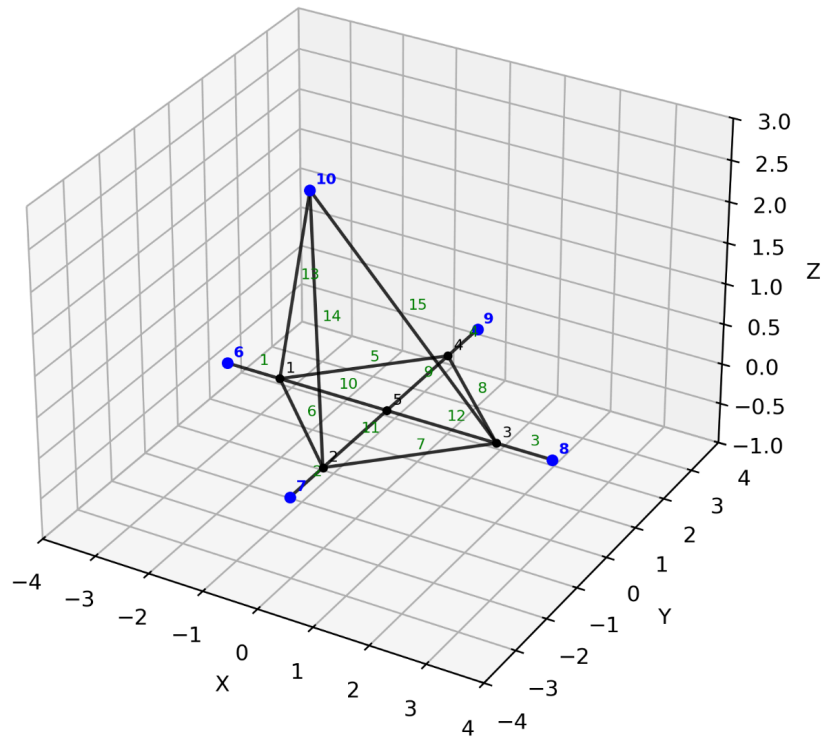
Structure 1 — Schek Structure with an Extra Node

Add one extra node out of plane in the z -direction, connected to the structure by three additional members.

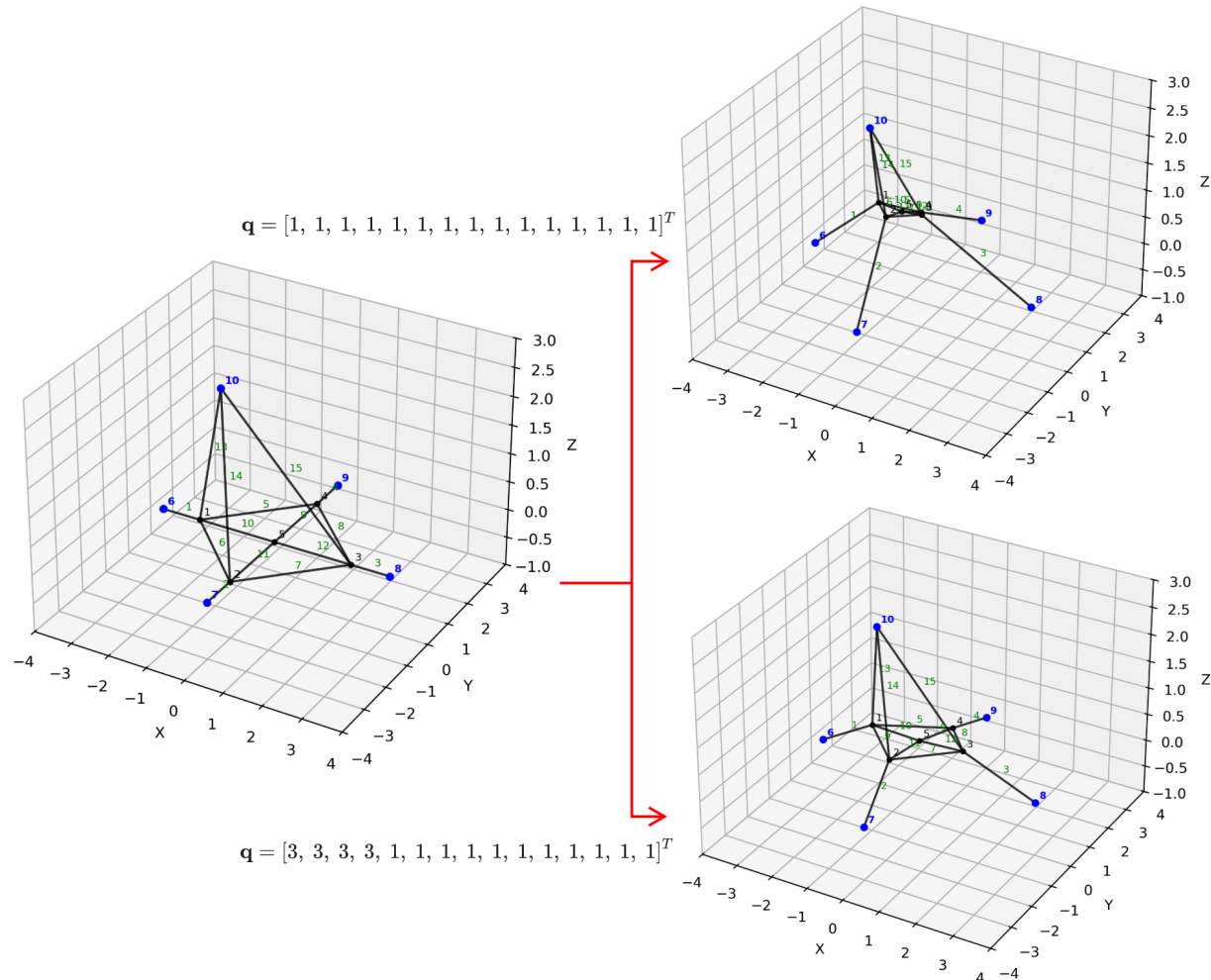
$$\mathbf{q} = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]^T$$

$$\mathbf{q} = [3, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]^T$$

Description of Structure



Results (Start -> Final)



Structure 2 — Veenendaal Validation Structure

A hyperbolic paraboloid cable-net with fixed boundaries is used as a validation example to compare all three form-finding methods:

- stiffness matrix method
- dynamic relaxation (with damping)
- force density method

The internal 3×3 mesh has 27 total DOFs.

All analyses start from an initially flat configuration.

Assumptions: $Q = 1$

Post-processing: Forces are normalized for comparison:

$$f_{\text{norm}} = \frac{f}{F_{\text{avg}}}$$

3. Minimal surface

As an example of the framework presented here, the stiffness matrix, dynamic relaxation with viscous and kinetic damping [9] and force density methods are compared by form finding a hyperbolic paraboloid cable-net (Figure 1) with fixed boundaries. Three mesh sizes are computed, with an increasing number of degrees of freedom (3 per node):

$n=3 \times 3$, 27 dof's (Figure 1), $n=7 \times 7$, 147 dof's and $n=15 \times 15$, 675 dof's. The form finding starts from an initially flat net (Table 2).

As a first trial, trivial input parameters are used: $Q=1$, $EA=F_0=1$ and all unstressed lengths L_0 are equal. The solution is considered to have converged when the squared sum of all residual forces changes by less than 1% between iterations, or when the residual forces are 0. The forces in the results are normalized to their average for comparison ($\mathbf{f}_{\text{norm}} = \mathbf{f}/F_{\text{avg}}$).

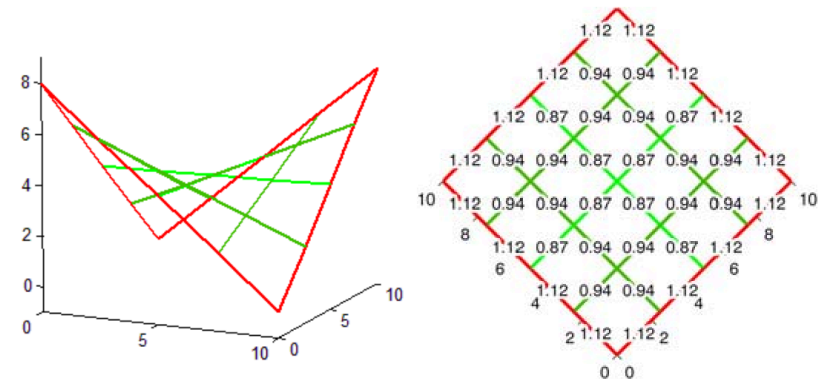
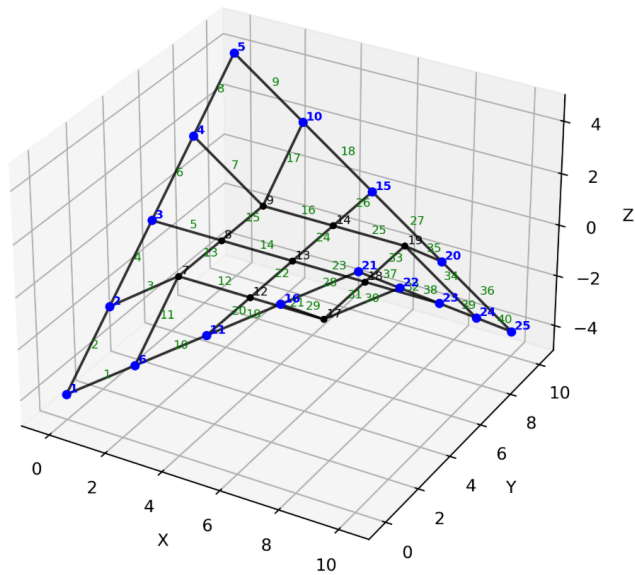
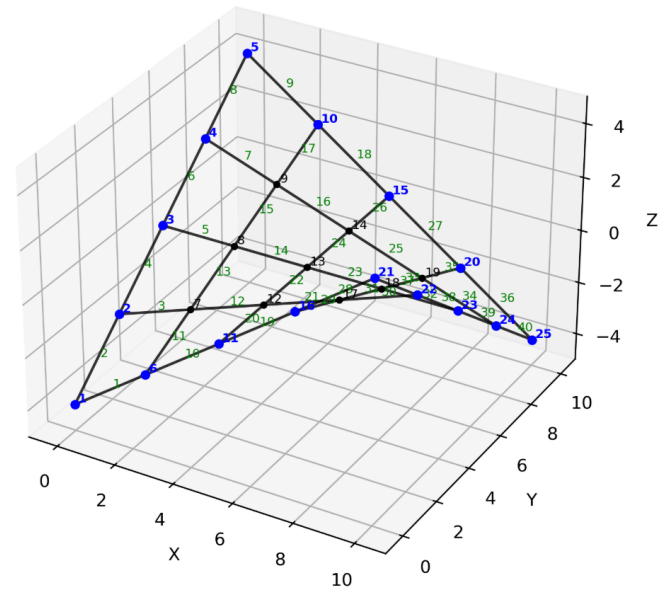


Fig. 1: Normalized forces $\mathbf{f}_{\text{norm}}$ for $Q=1$, or $EA=F_0=1$

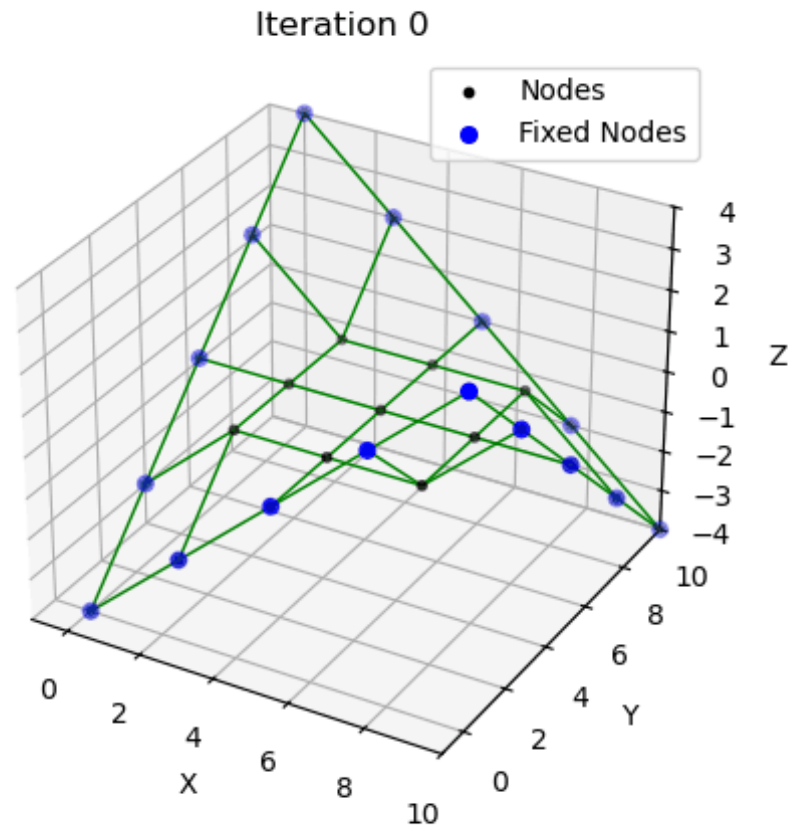
Results (Start -> Final)



$Q = 1$



Animation (Only Visible in Jupyter Notebook)



Structure 3 — Pauletti Validation Structure

Validation example from:

Pauletti & Fernandes (2020) — *An outline of the natural force density method and its extension to quadrilateral elements*

(see L13 folder for the paper)

A dense cable net with pinned supports at opposite corner nodes.

The geometry is extended to 3D by offsetting the corners in the z -direction.

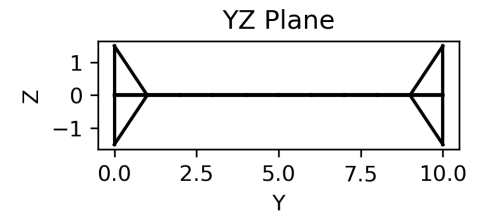
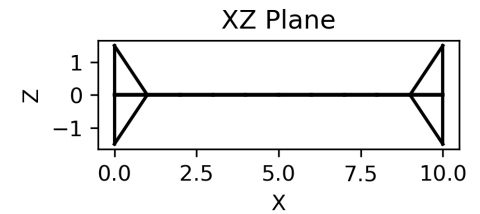
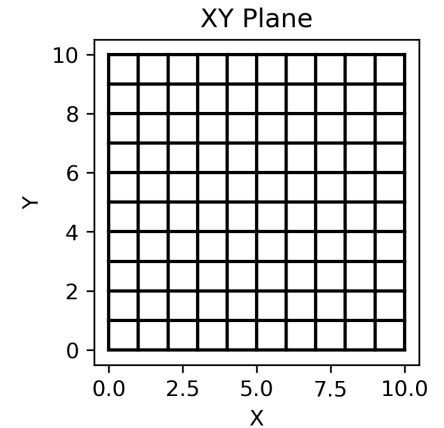
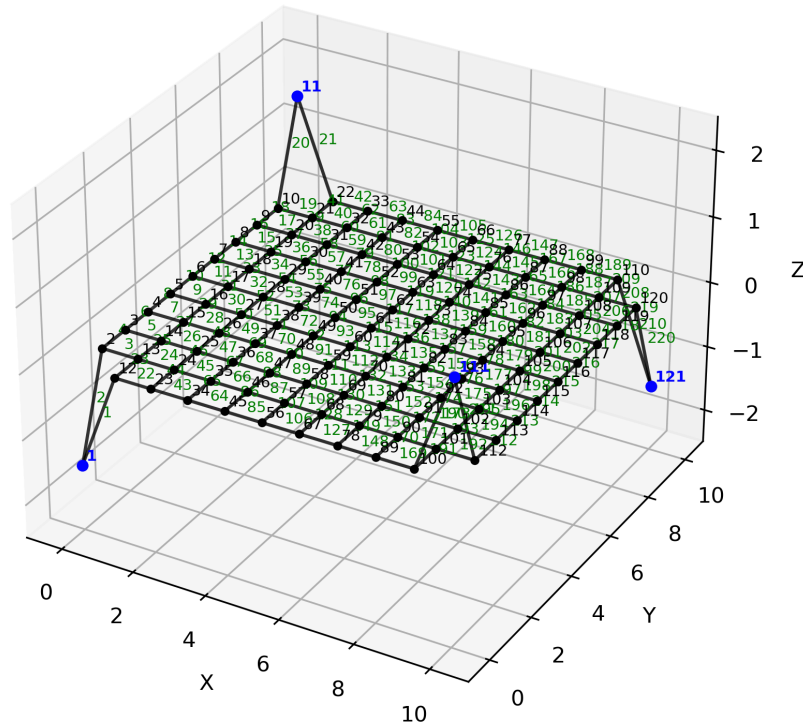
This setup is similar to the 2D "Complex Net", but introduces out-of-plane form finding.

Study parameter:

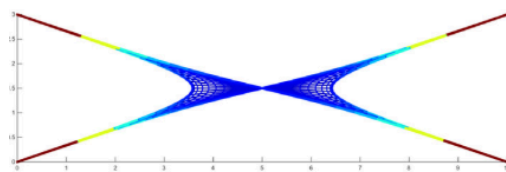
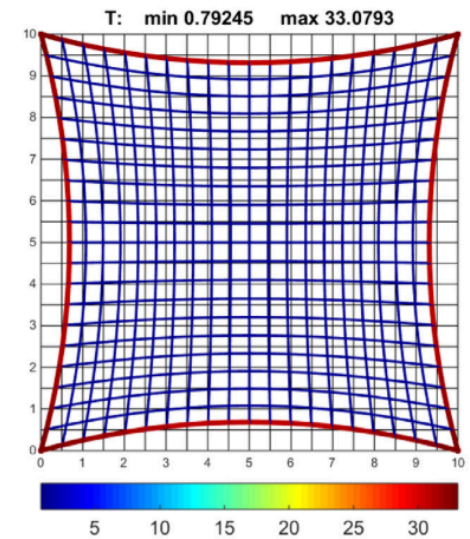
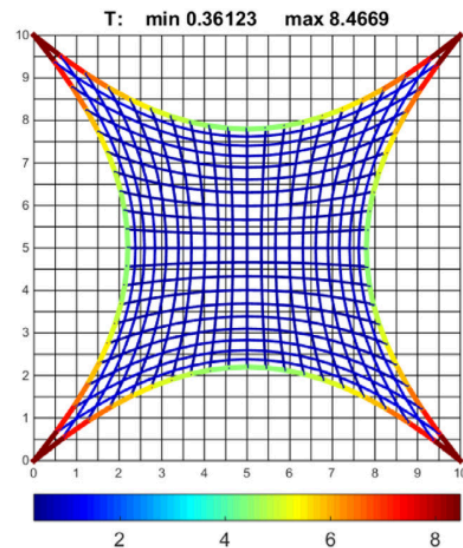
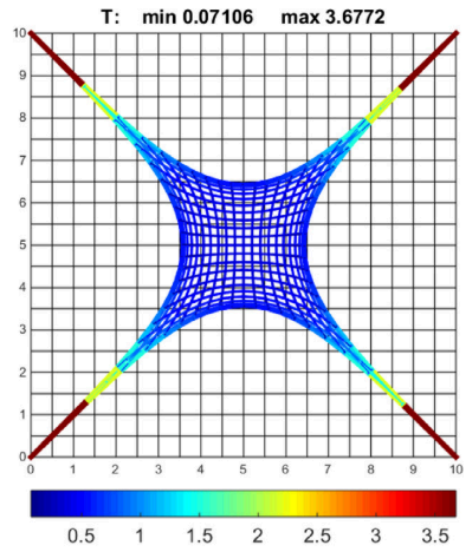
- Ratio of outer to inner force densities:

$$q_{\text{outer}} : q_{\text{inner}} = 1 : 1, 5 : 1, 30 : 1$$

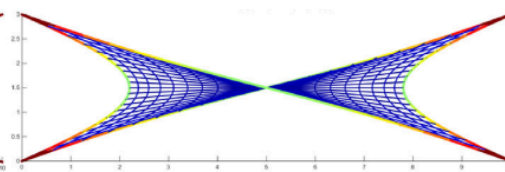
Description of Structure



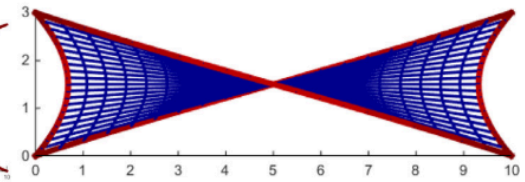
Results from Pauletti Paper



$$N_b = 1; N_i = 1$$

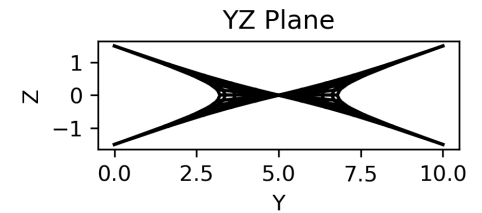
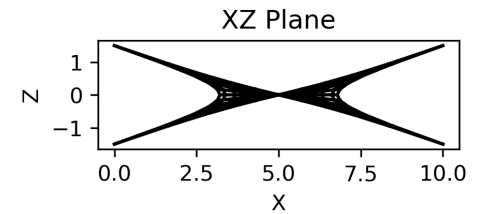
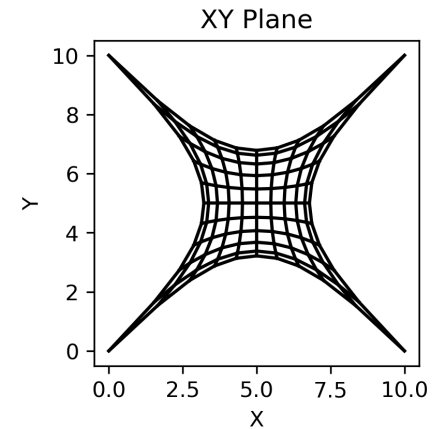
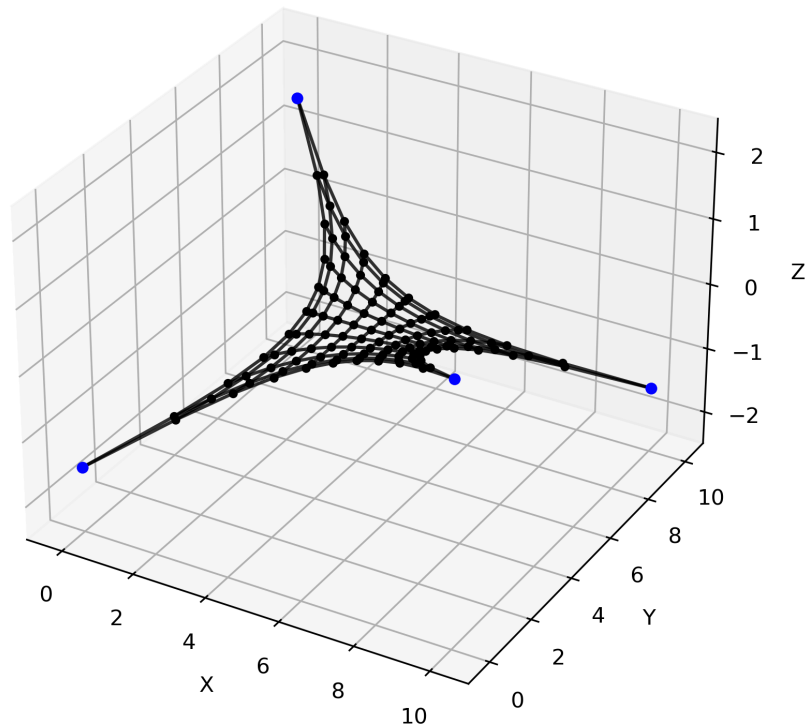


$$N_b = 5; N_i = 1$$

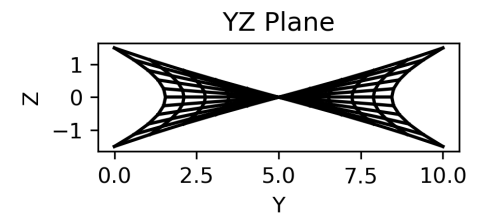
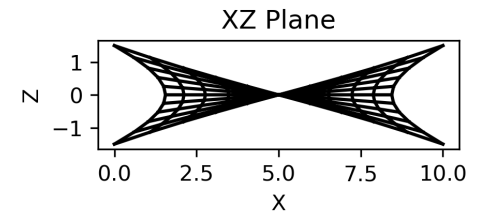
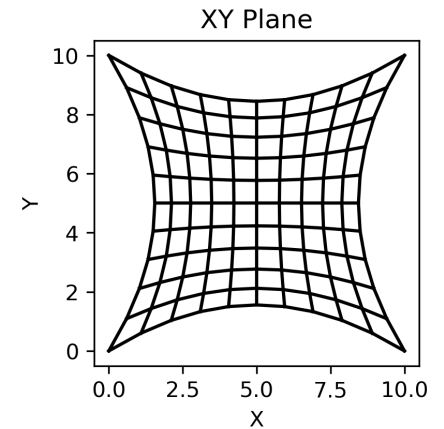
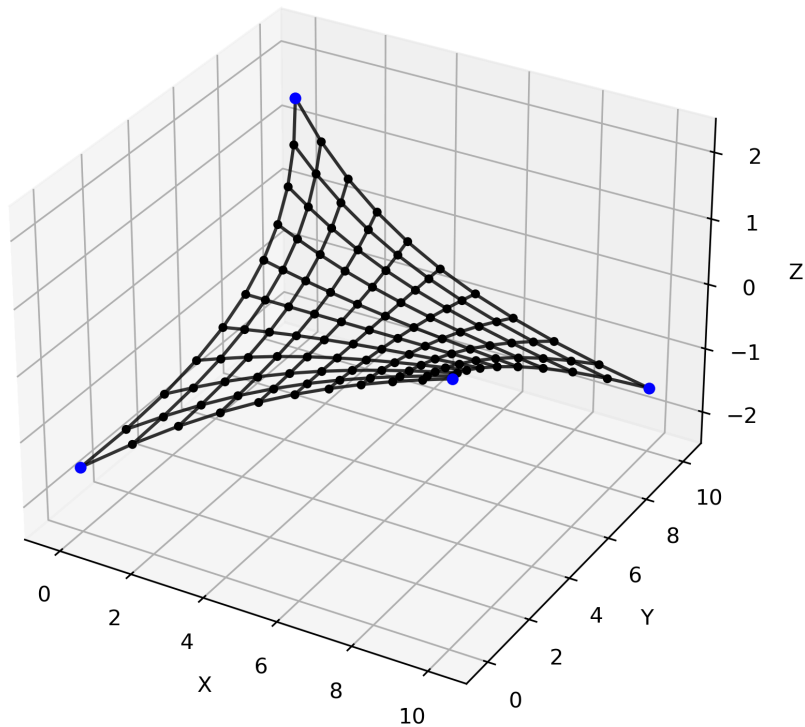


$$N_b = 30; N_i = 1$$

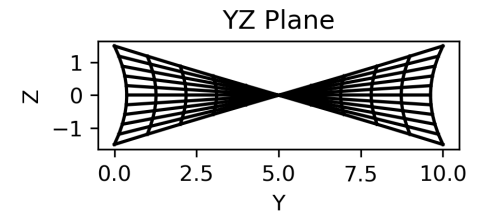
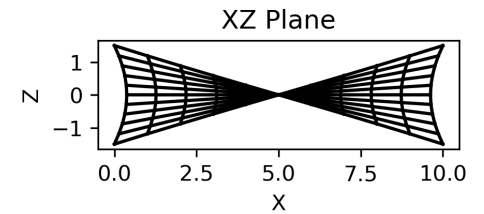
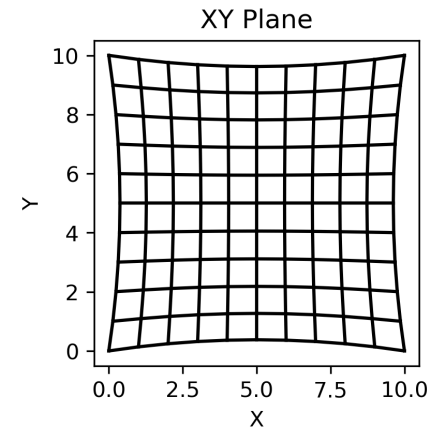
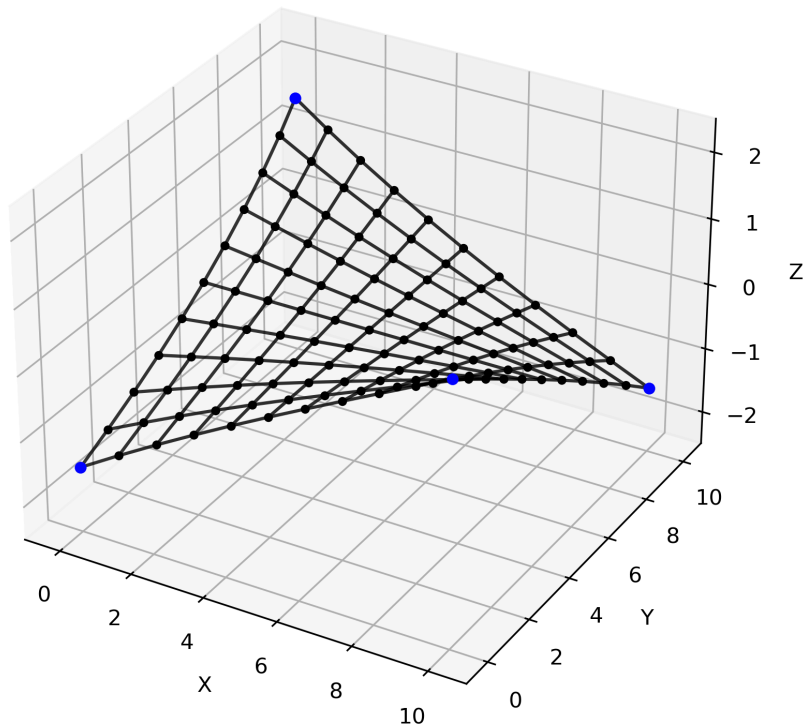
Result from 1:1 Case



Result from 5:1 Case



Result from 30:1 Case



Part 6 - Adding Applied Loads

Structure 1 - CEE6501 Point Loaded Structure

So far, we have only considered form-finding problems in which the internal force state is prescribed through the force density values. You can think of this as a specified prestress or pretension in the network.

We can also include external applied loads, so the structure must now find a geometry that satisfies equilibrium under both:

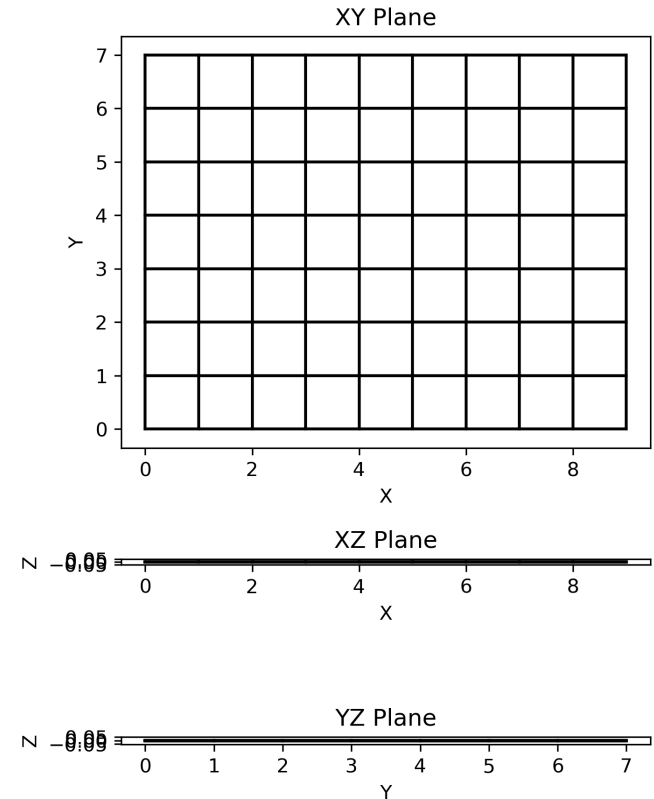
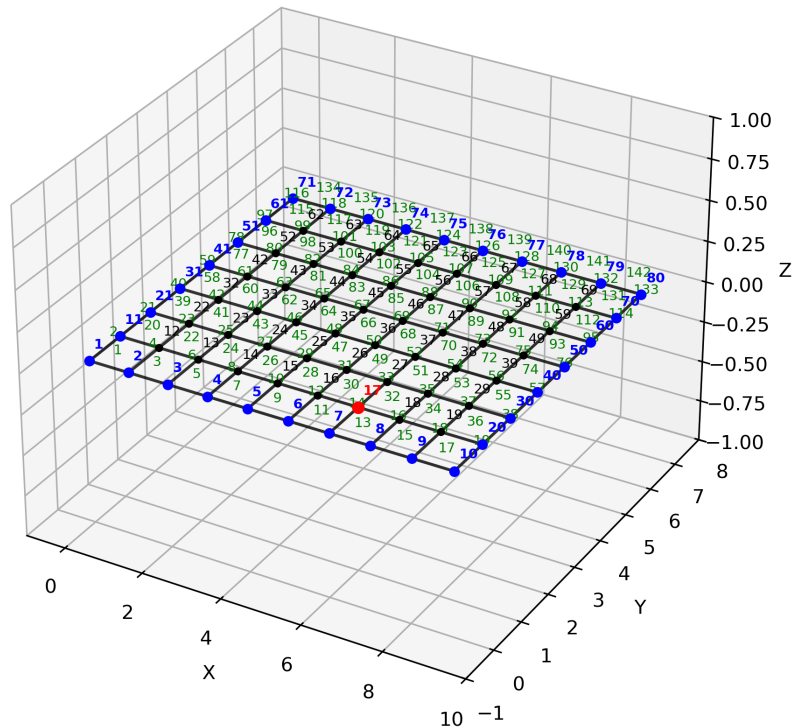
- the prescribed internal force density state, and
- the external nodal loads

Question:

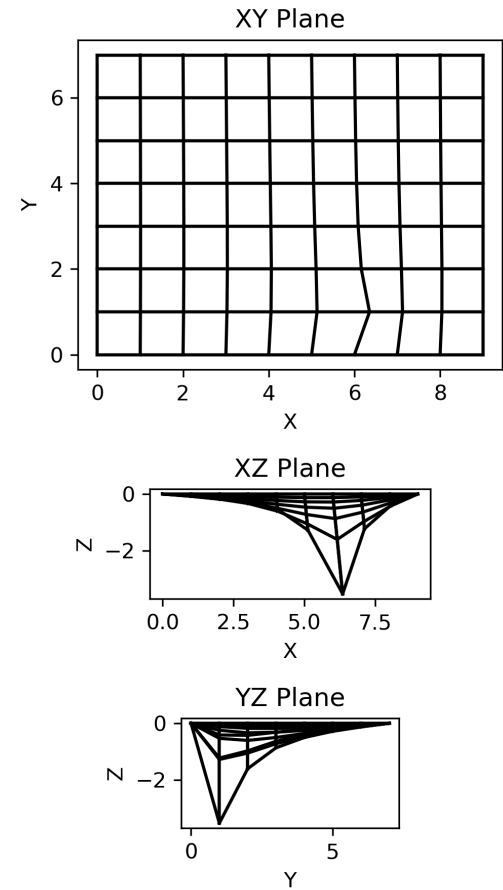
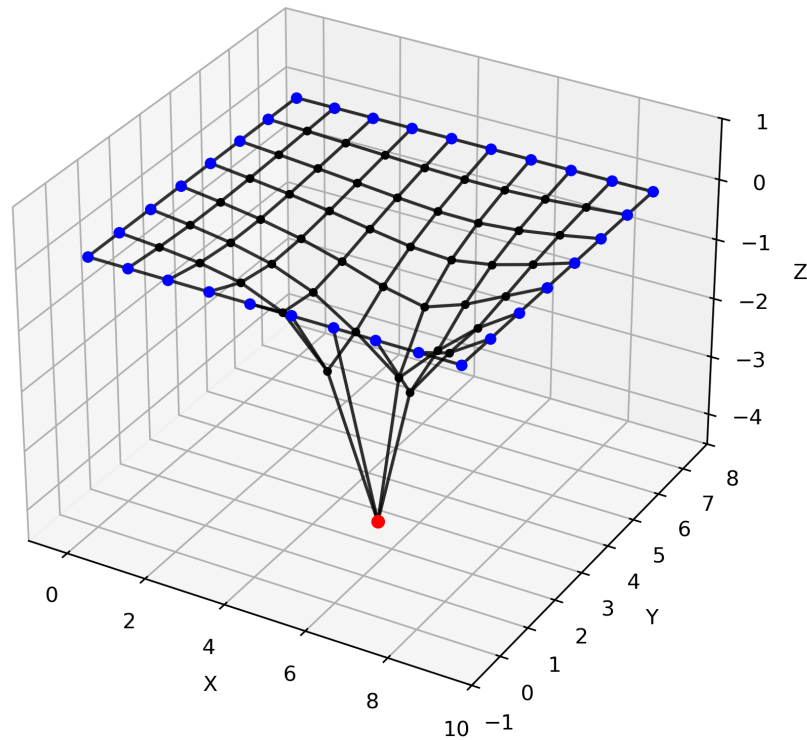
What shape does the net take under these combined conditions?

In this example, we prescribe a uniform force density, $q_i = 1$, in all members, and also apply downward point loads at selected nodes.

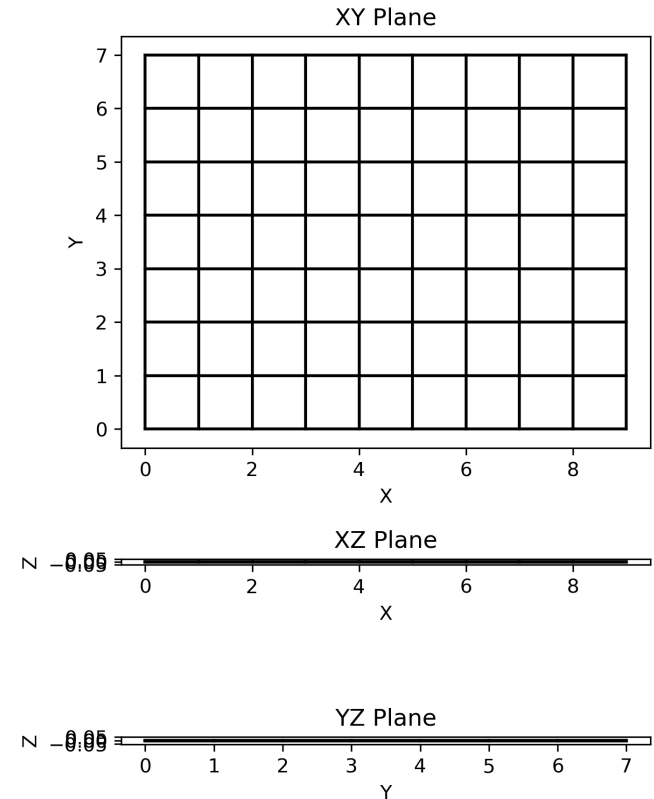
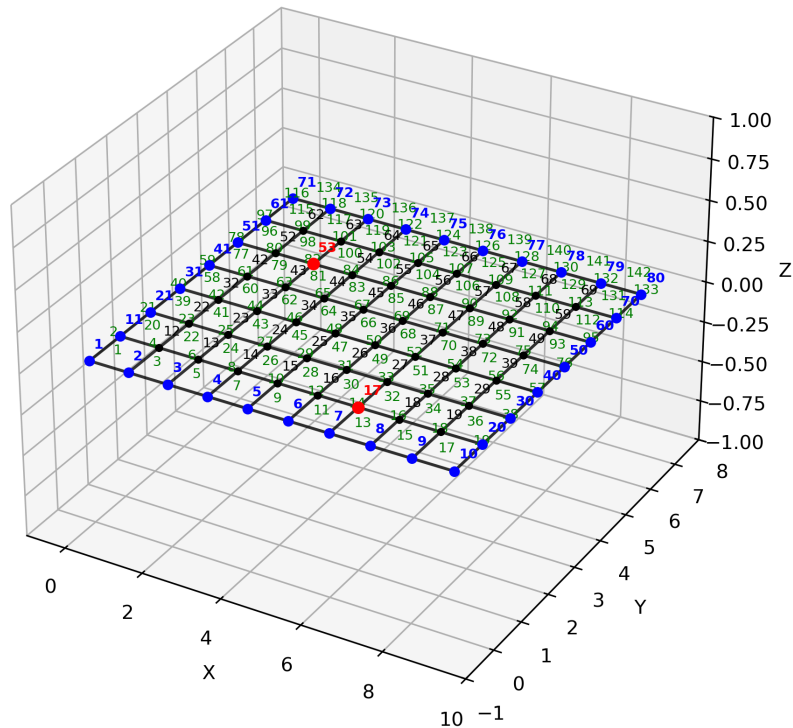
Single Point Load (2.0, 0.0, -10.0) at node 17



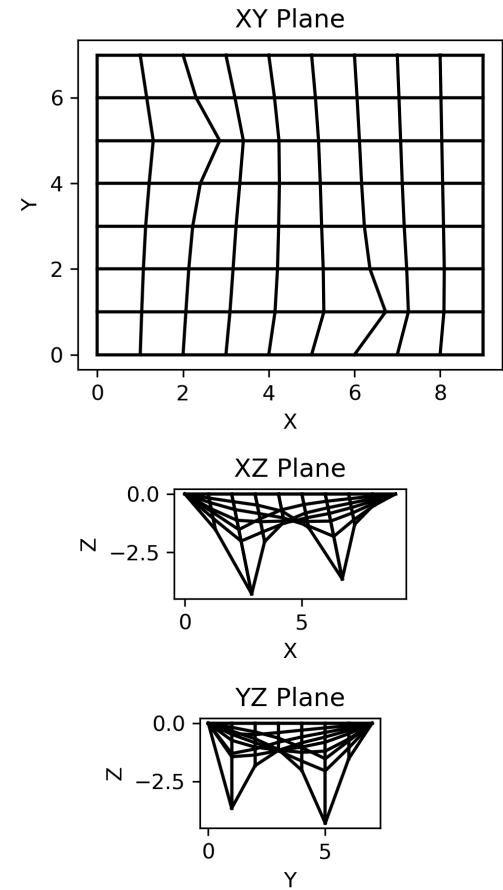
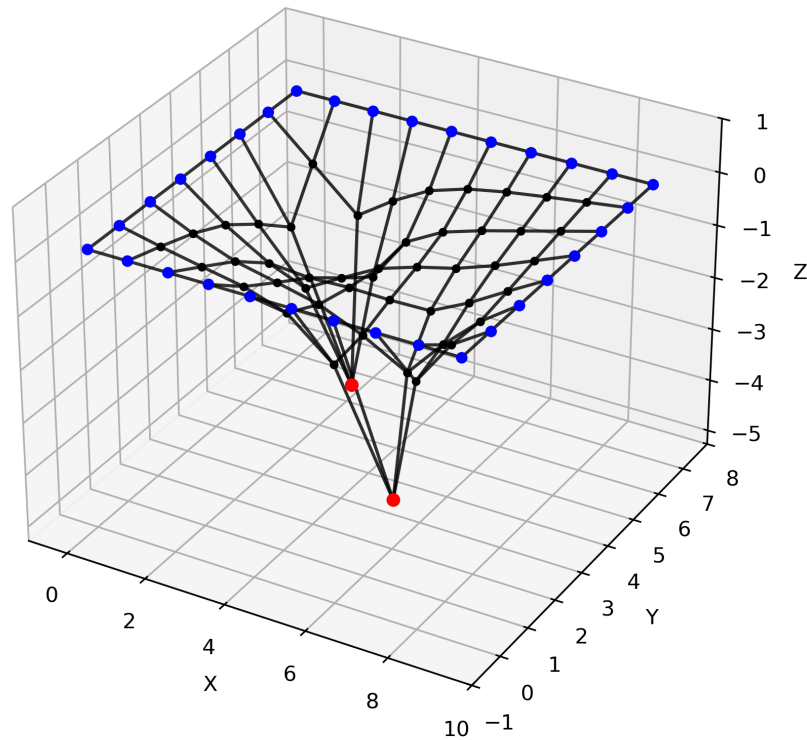
Result from Single Point Case



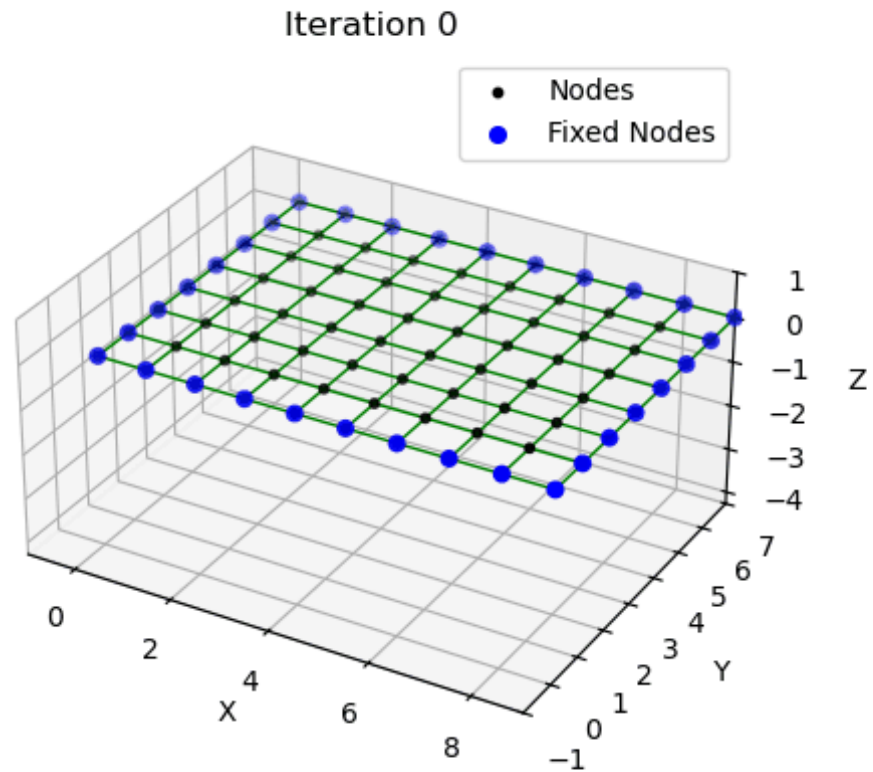
Two Point Loads (2.0, 0.0, -10.0) at node 17 and 53



Result from Double Point Case



Animation (Only Visible in Jupyter Notebook)



Wrap-Up

In this lecture, we:

- reviewed the linear force density formulation
- saw how equilibrium geometry is obtained through a single linear solve
- implemented the method in code and explored different structures
- studied how relative force densities control the resulting form
- extended the method to 3D and to cases with applied loads

Main takeaway:

Once connectivity, supports, loads, and force densities are prescribed, the equilibrium geometry follows directly from a linear system.

Next Lecture

Force Density Method (Nonlinear + Constraints) and comparison to other form-finding methods (SM and DR)